

¿Cómo eliminar una Cola ?

Se debe emplear la función `vQueueDelete()` para eliminar una cola.

A continuación se detallan los aspectos técnicos para su implementación:

1. Archivos de cabecera y Prototipo

Para utilizar esta función, es necesario incluir las siguientes librerías en tu archivo de código fuente:

C

```
#include "FreeRTOS.h"
#include "queue.h"
```

```
void vQueueDelete( QueueHandle_t xQueue );
```

(Nota: En algunas secciones de la documentación o versiones históricas, el parámetro de la firma puede aparecer genéricamente nombrado como `TaskHandle_t` `pxQueueToDelete`, pero la función está diseñada para recibir el manejador de la cola).

2. Funcionamiento y Características

- **Liberación de memoria:** Esta función destruye la cola y libera automáticamente toda la memoria RAM asignada dinámicamente en el *Heap* durante su creación (tanto el bloque de control de la cola como el espacio de almacenamiento reservado para sus elementos).
- **Valor de retorno:** No devuelve ningún valor (`void`).

Advertencia de Seguridad Crítica

- **Tareas en estado de bloqueo (Blocked):** No se debe eliminar una cola bajo ninguna circunstancia si existen tareas activas que se encuentren bloqueadas esperando en ella. Esto aplica tanto a tareas que esperan para leer datos de una cola vacía, como a tareas esperando para escribir en una cola llena. Borrar la cola en este escenario causará comportamientos totalmente impredecibles o fallos críticos en el kernel del sistema. Asegúrate siempre de liberar o finalizar dichas tareas de antemano.

¿Cómo crear una Cola ?

Para crear una cola (*Queue*) se debe utilizar la función `xQueueCreate()`.

En FreeRTOS, las colas almacenan los datos **por copia** (no por referencia). Esto significa que cuando envías un elemento a la cola, el contenido completo del dato se copia dentro del almacenamiento interno de la cola.

1. El Prototipo de `xQueueCreate()`

Para tener acceso a las funciones de colas, debes incluir siempre `queue.h`.

C

```
#include "FreeRTOS.h"
#include "queue.h"
```

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength,
                             UBaseType_t uxItemSize );
```

Parámetros:

- **uxQueueLength:** El número máximo de elementos que la cola puede contener al mismo tiempo.
- **uxItemSize:** El tamaño (en bytes) de cada elemento de la cola. Se recomienda utilizar el operador `sizeof()` para evitar errores de cálculo manual.

Valor de Retorno:

- Devuelve un manejador del tipo `QueueHandle_t`.
- Si el retorno es `NULL`, significa que la cola no se pudo crear porque **no hay suficiente memoria RAM libre** en el *Heap* de FreeRTOS.

💡 Buenas Prácticas y Consejos Técnicos

- **Verificación Obligatoria:** Nunca asumas que `xQueueCreate()` funcionará siempre. Si modificas tu código en el futuro y te quedas sin espacio en el *Heap*, la función devolverá `NULL` y tu programa fallará (*Hard Fault*) si intentas usar la cola.
- **El tamaño importa (uxItemSize):** Como FreeRTOS copia los datos enteros dentro de la cola, si necesitas enviar estructuras gigantescas (como un búfer de red de 1KB), **no metas la estructura pesada en la cola**. En su lugar, crea la cola para almacenar **punteros** (`sizeof(void*)`) que apunten a la memoria donde reside el dato.
- **Inicialización global:** Inicializar el manejador (`QueueHandle_t`) en `NULL` te ayuda a verificar en otras partes del código si la cola ya está lista para ser utilizada (`if (xColaComandos != NULL)`).

¿Cómo gestiona una Cola los datos que contiene?

En la API nativa de FreeRTOS, la gestión interna de los datos que contiene una cola se rige por varios principios arquitectónicos fundamentales. Comprenderlos es clave para optimizar el rendimiento y evitar fallos de memoria.

A continuación, se detalla paso a paso cómo administra FreeRTOS los datos en una cola de manera interna:

1. El Principio Fundamental: Almacenamiento “Por Copia” (Pass-by-Copy)

A diferencia de otros sistemas operativos que pasan punteros o referencias, **FreeRTOS gestiona las colas copiando el contenido real de los datos**.

- **Al enviar datos (xQueueSend / xQueueSendToBack / xQueueSendToFront):** El kernel toma la dirección de memoria de la variable que le pases (`pvItemToQueue`) y realiza un copiado binario directo (`memcpy`) de tantos bytes como se hayan definido en la

creación de la cola hacia el almacenamiento interno de la misma.

- **Al recibir datos (xQueueReceive / xQueuePeek):** El kernel copia los bytes guardados internamente en la cola hacia el búfer en RAM provisto por la tarea receptora (pvBuffer).

Implicaciones de este diseño:

- **Ventaja:** La tarea emisora puede reutilizar o destruir inmediatamente la variable local o estructura original apenas la función de envío termine, ya que la cola ya posee una copia idéntica e independiente del dato.
- **Desventaja (Rendimiento):** Copiar estructuras de datos muy grandes (búferes de red, imágenes, etc.) consume mucho tiempo de CPU.
- **Solución para datos grandes:** En lugar de guardar la estructura gigante en la cola, **se configuran las colas para almacenar punteros**. Al enviar, se copia solo la dirección de memoria (4 u 8 bytes dependiendo de la arquitectura), agilizando enormemente el proceso.

2. Organización de los datos: FIFO y LIFO

Internamente, la memoria intermedia asignada a la cola se organiza de forma lineal circular, permitiendo dos tipos de inserciones:

- **FIFO (First In, First Out - Por defecto):** Las funciones xQueueSend() o xQueueSendToBack() colocan el nuevo elemento al **final** (cola) del almacenamiento. La tarea que lee siempre extraerá el dato más antiguo que ingresó.
- **LIFO (Last In, First Out - Opcional):** La función xQueueSendToFront() coloca el nuevo elemento al **principio** (frente) del almacenamiento. El próximo xQueueReceive() leerá de inmediato este dato, saltándose el orden de llegada. Esto resulta muy útil para enviar notificaciones de emergencia o mensajes de alta prioridad.

3. Memoria Estática y Homogénea

Cuando creas una cola de forma nativa con xQueueCreate(), especificas dos parámetros inmutables durante el ciclo de vida de la cola:

1. **uxQueueLength:** Capacidad máxima de elementos.
2. **uxItemSize:** Tamaño en bytes de cada elemento.

A partir de estos datos, FreeRTOS reserva un **único bloque contiguo de memoria RAM** en el *Heap* proporcional a $uxQueueLength * uxItemSize$.

- **Homogeneidad:** Todos los elementos dentro de una misma cola deben medir exactamente lo mismo. No puedes meter un entero (int) y luego una estructura compleja en la misma cola si sus tamaños difieren.

4. Sincronización y Seguridad de Hilos (*Thread-Safety*)

Una cola nativa de FreeRTOS no solo almacena los datos, sino que internamente gestiona dos listas de tareas (*Task Lists*) que se encargan de la sincronización:

- **Lista de tareas bloqueadas por lectura (xTasksWaitingToReceive):** Si una tarea intenta leer de una cola vacía especificando un tiempo de espera (*Block Time*), el kernel la remueve de la lista de tareas listas (*Ready*) y la coloca en esta lista de bloqueo. En el instante en que otra tarea deposita un dato, el kernel despierta automáticamente a la tarea de mayor prioridad que estaba esperando.
- **Lista de tareas bloqueadas por escritura (xTasksWaitingToSend):** Si una tarea intenta escribir en una cola que ya está llena, se bloquea y se añade a esta lista. Tan pronto como otra tarea lee un elemento y libera espacio, el kernel despierta a la tarea escritora para que culmine el copiado de su dato.

Gracias a estos mecanismos internos protegidos por regiones críticas y deshabilitación temporal de interrupciones, las colas son completamente seguras de utilizar de forma concurrente por múltiples tareas simultáneas.

¿Cómo enviar datos a una Cola ?

Para enviar datos a una cola utilizando la **API nativa de FreeRTOS**, dispones de varias funciones. La elección de la función correcta depende de dos factores: el **orden en el que quieres encolar el dato** (FIFO o LIFO) y **desde dónde** estás llamando a la función (desde una tarea normal o desde una Rutina de Servicio de Interrupción - ISR).

Como se mencionó anteriormente, FreeRTOS copia el dato completo dentro de la cola (almacenamiento por copia).

1. Enviar datos desde una Tarea (Task)

Si estás dentro del flujo normal de una tarea, tienes tres funciones principales. Todas ellas son seguras para hilos (*thread-safe*) y permiten especificar un tiempo de espera si la cola está llena.

A. xQueueSend() o xQueueSendToBack() (Orden FIFO - El estándar)

Inserta el elemento al **final** de la cola. xQueueSend() y xQueueSendToBack() son exactamente equivalentes (la primera es un macro de la segunda).

C

```
BaseType_t xQueueSend( QueueHandle_t xQueue,
                      const void * pvItemToQueue,
                      TickType_t xTicksToWait );
```

B. xQueueSendToFront() (Orden LIFO - Mensajes urgentes)

Inserta el elemento al **principio** de la cola. Será el próximo elemento en ser leído, saltándose a todos los que ya estaban esperando.

C

```
BaseType_t xQueueSendToFront( QueueHandle_t xQueue,
                             const void * pvItemToQueue,
                             TickType_t xTicksToWait );
```

Explicación de los Parámetros:

- **xQueue:** El manejador de la cola a la que envías el dato.

- **pvItemToQueue:** Un puntero al dato que quieres enviar. La función tomará los bytes correspondientes al tamaño configurado de la cola y los copiará.
- **xTicksToWait:** El tiempo máximo que la tarea debe permanecer en estado *Blocked* esperando a que se libere espacio si la cola está llena.
 - 0: No espera; si está llena, regresa inmediatamente.
 - portMAX_DELAY: Espera indefinidamente (requiere que INCLUDE_vTaskSuspend esté en 1 en FreeRTOSConfig.h).
 - Cualquier otro valor (ej. pdMS_TO_TICKS(100)): Espera el tiempo indicado en milisegundos.

Valor de Retorno:

- **pdPASS:** El dato se copió con éxito en la cola.
- **errQUEUE_FULL:** La cola estaba llena y se agotó el tiempo de espera configurado sin que se liberara espacio.

2. Ejemplo Práctico desde una Tarea

A continuación, un ejemplo de cómo una tarea estructura y envía un dato (por ejemplo, lecturas de un sensor):

C

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

// Estructura de datos
typedef struct {
    uint8_t sensorID;
    float lectura;
} SensorData_t;

// Manejador global de la cola
extern QueueHandle_t xSensorQueue;

void vTareaSensor( void *pvParameters ) {
    SensorData_t datosEnvio;
    datosEnvio.sensorID = 2;

    for( ;; ) {
        // Leer el sensor simulado
        datosEnvio.lectura = 24.5f;

        // Enviar a la cola. Espera hasta 10 milisegundos si está llena.
        BaseType_t xStatus = xQueueSend( xSensorQueue, &datosEnvio,
pdMS_TO_TICKS(10) );

        if( xStatus == pdPASS ) {
            // Enviado correctamente
        } else {
```

```

        // Error: La cola se mantuvo llena por más de 10ms
    }

    vTaskDelay( pdMS_TO_TICKS(500) ); // Muestreo cada 500ms
}
}

```

3. Enviar datos desde una Interrupción (ISR)

Regla de Oro en FreeRTOS: No debes usar las funciones normales dentro de una interrupción de hardware. Para las ISR existen versiones específicas que terminan en `FromISR`. Estas funciones **no bloquean** (no tienen parámetro de tiempo de espera) y se ejecutan de manera determinista.

- `xQueueSendToBackFromISR()` (o `xQueueSendFromISR()`)
- `xQueueSendToFrontFromISR()`

Prototipo:

C

```

BaseType_t xQueueSendToBackFromISR( QueueHandle_t xQueue,
                                     const void * pvItemToQueue,
                                     BaseType_t * pxHigherPriorityTaskWoken );

```

El parámetro `pxHigherPriorityTaskWoken`:

Este puntero apunta a una variable que FreeRTOS pondrá en `pdTRUE` si el envío de este dato despierta a una tarea de mayor prioridad que la tarea actualmente interrumpida. Si esto pasa, debes solicitar un cambio de contexto manual antes de salir de la interrupción para que el procesador ejecute inmediatamente la tarea despertada.

Ejemplo en una ISR:

C

```

void EXTI0_IRQHandler( void ) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    uint32_t codigoEvento = 0xAA55;

    // Enviar dato desde la ISR
    xQueueSendFromISR( xEventoQueue, &codigoEvento, &xHigherPriorityTaskWoken
);

    // Si xHigherPriorityTaskWoken cambió a pdTRUE, forzamos el cambio de
    contexto
    // La macro exacta depende de la arquitectura (portYIELD_FROM_ISR o
    portEND_SWITCHING_ISR)
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

¿Cómo recibir datos de una Cola ?

Para recibir o extraer datos de una cola utilizando la **API nativa de FreeRTOS**, dispones principalmente de dos funciones: **xQueueReceive()** (para extraer el dato borrándolo de la cola) y **xQueuePeek()** (para mirar el dato sin borrarlo).

Al igual que al enviar, la forma de proceder varía si estás leyendo desde una tarea común o desde una rutina de interrupción (ISR).

1. Recibir datos desde una Tarea (Task)

Cuando ejecutas el código dentro de una tarea, utilizas **xQueueReceive()**. Esta función es bloqueante, lo que significa que si la cola está vacía, puedes configurar la tarea para que “duerma” automáticamente hasta que llegue un dato, optimizando el uso de la CPU.

Prototipo:

C

```
BaseType_t xQueueReceive( QueueHandle_t xQueue,
                          void * pvBuffer,
                          TickType_t xTicksToWait );
```

Parámetros:

- **xQueue**: El manejador de la cola de la cual vas a leer.
- **pvBuffer**: Puntero a la variable local de tu tarea donde se van a **copiar** los datos recibidos. Esta variable debe tener el tamaño exacto del elemento configurado al crear la cola.
- **xTicksToWait**: El tiempo máximo que la tarea esperará en estado *Blocked* si la cola está vacía.
 - 0: No espera; si está vacía, retorna inmediatamente con un error.
 - portMAX_DELAY: Espera indefinidamente hasta que otra tarea o ISR envíe un dato.
 - pdMS_TO_TICKS(ms): Espera el tiempo exacto especificado en milisegundos.

Valor de Retorno:

- **pdPASS**: Se leyó un elemento correctamente y se almacenó en tu búfer (pvBuffer). El elemento **es eliminado** de la cola.
- **pdFALSE** (o errQUEUE_EMPTY): No se pudo leer ningún elemento porque la cola estaba vacía y el tiempo de espera expiró.

2. Ejemplo Práctico en una Tarea

A continuación, un ejemplo de una tarea consumidora que procesa estructuras de datos provenientes de una cola de manera eficiente:

C

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
```

```
// Estructura idéntica a la que se usó para enviar
typedef struct {
    uint8_t sensorID;
    float lectura;
} SensorData_t;

extern QueueHandle_t xSensorQueue; // Manejador de la cola creado en otro
sitio

void vTareaConsumidora( void *pvParameters ) {
    SensorData_t datosRecibidos;

    for( ;; ) {
        // Bloquear de forma indefinida (portMAX_DELAY) hasta que llegue un
dato.
        // La tarea no consume CPU mientras está bloqueada aquí.
        if( xQueueReceive( xSensorQueue, &datosRecibidos, portMAX_DELAY ) ==
pdPASS ) {
            // El dato se copió con éxito en 'datosRecibidos'
            if( datosRecibidos.sensorID == 2 ) {
                // Procesar la lectura del sensor
                printf("Lectura Sensor 2: %.2f\n", datosRecibidos.lectura);
            }
        }
    }
}
```

3. ¿Qué pasa si solo quiero mirar el dato sin borrarlo? (xQueuePeek)

Si necesitas leer un dato para inspeccionarlo pero quieres que **permanezca en la cola** para que otra tarea (o la misma) lo procese formalmente más tarde, debes usar **xQueuePeek()**. Utiliza exactamente los mismos parámetros y lógica de bloques que xQueueReceive(), pero no altera el contenido de la cola.

C

```
// Copia el elemento del frente de la cola en 'datosRecibidos' pero no lo
elimina
xQueuePeek( xSensorQueue, &datosRecibidos, pdMS_TO_TICKS(10) );
```

4. Recibir datos desde una Interrupción (ISR)

Al igual que sucede con el envío, nunca debes llamar a xQueueReceive() o xQueuePeek() desde una rutina de interrupción de hardware. Las ISR requieren funciones especiales no bloqueantes que terminen en FromISR:

- xQueueReceiveFromISR()
- xQueuePeekFromISR()

Prototipo:

C

```
BaseType_t xQueueReceiveFromISR( QueueHandle_t xQueue,  
                                void * const pvBuffer,  
                                BaseType_t * pxHigherPriorityTaskWoken );
```

Si la lectura de la cola libera un espacio que permite que una tarea de alta prioridad (que estaba bloqueada intentando escribir en la cola llena) se despierte, el argumento `pxHigherPriorityTaskWoken` cambiará a `pdTRUE`. En ese caso, deberás forzar un cambio de contexto manual antes de salir de la ISR.

¿Qué significa bloquearse en una Cola ?

En la API nativa de FreeRTOS, **bloquearse en una cola** (*Queue Blocking*) es el mecanismo que permite a una tarea entrar en un estado de “espera suspendida” automáticamente cuando no puede completar una operación de lectura o escritura.

Mientras una tarea está en estado **Bloqueado (Blocked)**, consume **0% de tiempo de CPU**, dejando todo el poder de procesamiento del microcontrolador para otras tareas del sistema que sí estén listas para ejecutarse.

El bloqueo puede ocurrir por dos razones inversas:

1. Bloqueo por Lectura (Cola Vacía)

Ocurre cuando una tarea intenta extraer un elemento de la cola utilizando `xQueueReceive()`, pero la cola está vacía.

- **Comportamiento:** Si especificas un tiempo de espera (*Block Time*) mayor a cero, el *scheduler* de FreeRTOS remueve inmediatamente a esa tarea del estado *Ready* (Lista) y la mueve al estado *Blocked* (Bloqueada).
- **¿Cuándo se desbloquea?** La tarea volverá al estado *Ready* en cualquiera de estos dos eventos (lo que ocurra primero):
 1. **Llega un dato:** Otra tarea o una interrupción (ISR) envía un elemento a la cola. Si la tarea bloqueada es la de mayor prioridad en el sistema, interrumpirá la ejecución actual para procesar el dato de inmediato.
 2. **Se agota el tiempo (Timeout):** Pasa el tiempo máximo configurado sin que ingrese ningún dato. La función `xQueueReceive()` se despierta y retorna `pdFALSE`.

2. Bloqueo por Escritura (Cola Llena)

Ocurre cuando una tarea intenta enviar un elemento utilizando `xQueueSend()` (o similares), pero la cola ya alcanzó su capacidad máxima configurada (`uxQueueLength`).

- **Comportamiento:** Si la tarea tiene un tiempo de espera configurado, se bloquea para detener la producción de datos.
- **¿Cuándo se desbloquea?** Volverá a estar lista en cualquiera de estas dos condiciones:
 1. **Se libera espacio:** Otra tarea lee de la cola mediante `xQueueReceive()`, vaciando una ranura de almacenamiento. El kernel despierta a la tarea escritora para que termine de copiar su dato.
 2. **Se agota el tiempo (Timeout):** El tiempo asignado expira y la cola sigue llena. La función retorna `errQUEUE_FULL`.

El Parámetro que Controla el Bloqueo: xTicksToWait

El tiempo que una tarea está dispuesta a bloquearse se define en el tercer parámetro de las funciones de colas:

C

```
// Ejemplo con xQueueReceive
xQueueReceive( xMiCola, &buffer, xTicksToWait );
```

Puedes pasarle tres tipos de valores a este parámetro:

- **0 (No Bloqueante):** La tarea **no se bloqueará**. Si la cola está vacía al intentar leer (o llena al intentar escribir), la función regresa inmediatamente con un código de error. Es útil si la tarea tiene otras cosas urgentes que hacer.
- **Un valor en Ticks / Milisegundos (Bloqueo Acotado):** Utilizando la macro `pdMS_TO_TICKS(50)`, le indicas al sistema que permita a la tarea bloquearse por un máximo de 50 milisegundos.
- **portMAX_DELAY (Bloqueo Indefinido):** La tarea se quedará bloqueada para siempre hasta que la condición de la cola cambie (llegue un dato o se libere espacio). *Nota: Para usar esto, INCLUDE_vTaskSuspend debe estar definido como 1 en el archivo FreeRTOSConfig.h.*

¿Qué pasa si múltiples tareas se bloquean en la misma cola?

FreeRTOS gestiona esto de forma automática y ordenada mediante listas de prioridad:

- Si hay varias tareas bloqueadas esperando leer de una cola vacía, en el momento en que llega un dato, el kernel **despierta únicamente a la tarea que tenga la prioridad más alta**.
- Si todas las tareas bloqueadas tienen exactamente la misma prioridad, el kernel despertará a **la tarea que lleve más tiempo esperando** (orden FIFO de tareas).

¿Cómo bloquearse en varias Cola ?

En la API nativa de FreeRTOS, por diseño estándar, una tarea **no puede bloquearse directamente en múltiples colas individuales al mismo tiempo** usando funciones como `xQueueReceive()`. Si intentas llamar a `xQueueReceive()` en una cola, la tarea se detendrá ahí y no podrá evaluar la segunda cola hasta que la primera reciba un dato.

Sin embargo, FreeRTOS ofrece una característica nativa específicamente diseñada para resolver este problema: **Los Conjuntos de Colas o Queue Sets**.

Un *Queue Set* te permite agrupar varias colas (y/o semáforos) de manera que una tarea pueda bloquearse en el “conjunto” completo. Cuando cualquiera de las colas del conjunto recibe un dato, la tarea se despierta.

Paso a Paso: Cómo implementar un Queue Set

Para poder utilizar esta característica, debes asegurarte de que en tu archivo `FreeRTOSConfig.h` la siguiente constante esté activada:

C

```
#define configUSE_QUEUE_SETS    1
```

1. Crear las colas y el Queue Set

Primero creas las colas individuales normalmente y luego creas el conjunto con `xQueueCreateSet()`. El tamaño del conjunto debe ser la suma de las longitudes de todas las colas que va a contener.

2. Añadir las colas al conjunto

Utilizas `xQueueAddToSet()` para asociar cada cola al conjunto creado. **Regla de oro:** Las colas deben estar completamente vacías al momento de añadirlas al conjunto.

3. Bloquearse en el conjunto y leer

En lugar de leer la cola directamente, la tarea llama a `xQueueSelectFromSet()`. Esta función bloqueará a la tarea hasta que alguna de las colas del conjunto tenga datos disponibles. Cuando se despierta, la función **devuelve el manejador de la cola que recibió el dato**. Tras saber cuál es, realizas el `xQueueReceive()` normal sobre esa cola específica.

Ejemplo Práctico de Código Nativo

A continuación se muestra cómo una sola tarea procesa datos provenientes de dos colas distintas de manera simultánea sin perder tiempo de CPU:

C

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

// Manejadores de las colas y del conjunto
QueueHandle_t xColaSensores = NULL;
QueueHandle_t xColaTeclado = NULL;
QueueSetHandle_t xConjuntoColas = NULL;

void vTareaConsumidoraMultiCola( void *pvParameters ) {
    QueueSetMemberHandle_t xColaActiva;
    uint32_t valorSensor;
    char teclaPresionada;

    // 1. Crear las colas individuales
    xColaSensores = xQueueCreate( 5, sizeof( uint32_t ) );
    xColaTeclado = xQueueCreate( 5, sizeof( char ) );

    // 2. Crear el Queue Set (Capacidad combinada: 5 + 5 = 10)
    xConjuntoColas = xQueueCreateSet( 10 );

    if( ( xColaSensores != NULL ) && ( xColaTeclado != NULL ) && (
xConjuntoColas != NULL ) ) {
        // 3. Añadir las colas al conjunto antes de que reciban datos
        xQueueAddToSet( xColaSensores, xConjuntoColas );
        xQueueAddToSet( xColaTeclado, xConjuntoColas );

        for( ;; ) {
            // 4. Bloquearse en el conjunto completo de manera indefinida
```

```

        xColaActiva = xQueueSelectFromSet( xConjuntoColas, portMAX_DELAY
);

// 5. Evaluar cuál de las colas fue la que despertó a la tarea
if( xColaActiva == xColaSensores ) {
    // Leer de la cola de sensores (garantizado que no se
bloqueará aquí)
    xQueueReceive( xColaSensores, &valorSensor, 0 );
    // Procesar valor del sensor...
}
else if( xColaActiva == xColaTeclado ) {
    // Leer de la cola de teclado
    xQueueReceive( xColaTeclado, &teclaPresionada, 0 );
    // Procesar tecla...
}
}
}
vTaskDelete( NULL );
}

```

Alternative Técnica más Eficiente: Estructuras Unificadas

Aunque los *Queue Sets* son la solución directa para escuchar múltiples colas, la propia documentación de FreeRTOS menciona que introducen una pequeña sobrecarga de rendimiento.

Si tienes control sobre el diseño del software, una alternativa sumamente común y eficiente es **usar una única cola que reciba una estructura con un “ID de Evento”**. De esta manera, solo necesitas una cola estándar, simplificando enormemente el código:

C

```

typedef enum {
    EVENTO_SENSOR,
    EVENTO_TECLADO
} TipoEvento_t;

typedef struct {
    TipoEvento_t tipo;
    union {
        uint32_t valorSensor;
        char tecla;
    } datos;
} MensajeUnificado_t;

// La tarea solo se bloquea en una cola de tipo MensajeUnificado_t
// y usa un switch(mensaje.tipo) para decidir qué hacer.

```

¿Cómo sobrescribir datos en una Cola ?

Para sobrescribir datos en una cola utilizando la API nativa de FreeRTOS, se utiliza la función **xQueueOverwrite()**.

Esta función está diseñada específicamente para colas que tienen una **longitud máxima de 1 elemento** (`uxQueueLength = 1`). Si la cola ya contiene un dato, lo destruye y lo reemplaza inmediatamente por el nuevo, evitando que la tarea emisora se bloquee.

1. El Prototipo de xQueueOverwrite()

Al igual que las demás funciones de colas, se encuentra en `queue.h`:

C

```
#include "FreeRTOS.h"
#include "queue.h"
```

```
BaseType_t xQueueOverwrite( QueueHandle_t xQueue,
                           const void * pvItemToQueue );
```

Parámetros:

- **xQueue**: El manejador de la cola donde se va a escribir.
- **pvItemToQueue**: Puntero al nuevo dato que va a copiar y sobrescribir en la cola.

Valor de Retorno:

- **pdPASS**: Siempre devuelve `pdPASS`. Como la función está obligada a escribir (ya sea vaciando el espacio o pisando el dato anterior), nunca se bloquea y nunca falla por “cola llena”.

2. Cuándo se usa (Casos de éxito típicos)

El uso de `xQueueOverwrite()` es ideal para variables de estado o lecturas de sensores continuas donde **solo importa el último valor conocido**.

- *Ejemplo*: Si un sensor lee la temperatura cada 10ms, a la tarea que grafica en pantalla no le importan los 50 valores anteriores que no llegó a procesar, solo le interesa pintar el *último* valor disponible.

3. Ejemplo Práctico de Implementación

A continuación, un ejemplo completo de cómo crear la cola de un solo elemento y sobrescribir su contenido:

C

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
```

```
// El manejador de la cola
QueueHandle_t xColaTemperatura = NULL;
```

```
void vTareaProductoraSensor( void *pvParameters ) {
    float temperaturaActual;
```

```
    for( ;; ) {
        // Leer sensor (supongamos que devuelve un valor que cambia)
        temperaturaActual = 25.4f;
```

```
        // Sobrescribir el dato. Si la tarea consumidora no lo ha leído,
```

```

        // no importa, pisamos el viejo con este nuevo valor de 25.4f.
        xQueueOverwrite( xColaTemperatura, &temperaturaActual );

        vTaskDelay( pdMS_TO_TICKS(100) ); // Muestreo cada 100ms
    }
}

void vTareaConsumidoraDisplay( void *pvParameters ) {
    float temperaturaRecibida;

    for( ;; ) {
        // Leer de la cola esperando indefinidamente si se vacía
        if( xQueueReceive( xColaTemperatura, &temperaturaRecibida,
portMAX_DELAY ) == pdPASS ) {
            // Imprime siempre el valor más fresco disponible
            printf("Temp en pantalla: %.1f \n", temperaturaRecibida);
        }
    }
}

int main( void ) {
    // REGLA CRÍTICA: La longitud DEBE ser 1 para usar xQueueOverwrite
    xColaTemperatura = xQueueCreate( 1, sizeof( float ) );

    if( xColaTemperatura != NULL ) {
        xTaskCreate( vTareaProductoraSensor, "Sensor", 128, NULL, 2, NULL );
        xTaskCreate( vTareaConsumidoraDisplay, "Display", 128, NULL, 1, NULL
    );

        vTaskStartScheduler();
    }
    for( ;; );
}

```

4. Sobrescribir datos desde una Interrupción (ISR)

Si necesitas hacer esta operación de sobrescritura dentro de una rutina de interrupción de hardware, FreeRTOS provee la variante segura para ISR:

C

```

 BaseType_t xQueueOverwriteFromISR( QueueHandle_t xQueue,
                                     const void * pvItemToQueue,
                                     BaseType_t * pxHigherPriorityTaskWoken );

```

Funciona exactamente igual, con la diferencia de que incluye el puntero `pxHigherPriorityTaskWoken` para gestionar cambios de contexto inmediatos si la sobrescritura despierta a una tarea de mayor prioridad que estaba esperando leer la cola.

Limitación técnica estricta

Si intentas usar `xQueueOverwrite()` o `xQueueOverwriteFromISR()` en una cola que fue creada con un tamaño mayor a 1 (por ejemplo, `xQueueCreate(5, sizeof(int))`), **el sistema lanzará un assert y el programa fallará**. El código interno de FreeRTOS valida estrictamente que `pxQueue->uxLength == 1`.

¿Cómo vaciar una Cola ?

Para vaciar una cola por completo en la API nativa de FreeRTOS (es decir, eliminar todos los elementos almacenados en ella y dejarla en su estado inicial de “vacía”), dispones de dos enfoques principales: **reinicializarla instantáneamente** o **extraer los elementos uno a uno**.

A continuación, te explico cómo implementar cada método.

Método 1: El enfoque rápido (Resetear la cola)

La forma más eficiente y elegante de vaciar una cola en FreeRTOS es utilizando la función **xQueueReset()**. Esta función restablece internamente los punteros de lectura y escritura de la cola a su posición inicial, dejándola con cero elementos de forma inmediata.

Prototipo:

C

```
#include "FreeRTOS.h"
#include "queue.h"
```

```
BaseType_t xQueueReset( QueueHandle_t xQueue );
```

- **Valor de retorno:** En las versiones modernas de FreeRTOS, siempre devuelve pdPASS.
- **Ventaja:** Es una operación de tiempo constante $O(1)$. No importa si la cola tiene 1 o 500 elementos, los vacía al instante.

Ejemplo de uso:

C

```
extern QueueHandle_t xMiCola;

void vTareaLimpieza( void *pvParameters ) {
    for( ;; ) {
        // ... en algún punto decides que los datos acumulados ya no son
        válidos ...

        if( xMiCola != NULL ) {
            xQueueReset( xMiCola );
            // La cola ahora está completamente vacía y lista para recibir
            datos frescos
        }

        vTaskDelay( pdMS_TO_TICKS(1000) );
    }
}
```

Método 2: El enfoque manual (Vaciar consumiendo)

Si por alguna razón necesitas **inspeccionar, contar o procesar** los elementos descartados a medida que los vas borrando (o si simplemente prefieres no resetear las estructuras internas bruscamente), puedes implementar un bucle que extraiga los elementos de manera no bloqueante utilizando `xQueueReceive()` con un tiempo de espera de 0.

Ejemplo de uso:

C

```
extern QueueHandle_t xMiCola;
typedef struct { /* ... tu tipo de dato ... */ } MiDato_t;

void vVaciarColaManualmente( void ) {
    MiDato_t datoDescartado;
    uint32_t elementosBorrados = 0;

    if( xMiCola != NULL ) {
        // xQueueReceive con un TickToWait de 0 retornará pdFALSE
        // en el instante exacto en que la cola se quede vacía.
        while( xQueueReceive( xMiCola, &datoDescartado, 0 ) == pdPASS ) {
            elementosBorrados++;
            // Opcional: procesar o registrar el 'datoDescartado' antes de
que se pierda
        }
    }
}
```

Regla de Oro y Comportamiento con Tareas Bloqueadas

Al utilizar `xQueueReset()`, debes tener muy en cuenta qué pasa si hay tareas interactuando con la cola en ese momento:

- **Tareas bloqueadas por Escritura (Cola Llena):** Si tenías tareas de alta prioridad en estado *Blocked* esperando que la cola tuviera espacio libre para poder escribir, al ejecutar `xQueueReset()`, FreeRTOS **despertará automáticamente a esas tareas escritoras**. Como la cola se vació de golpe, estas tareas procederán de inmediato a copiar sus datos pendientes en ella.
- **Tareas bloqueadas por Lectura (Cola Vacía):** Si usas `xQueueReset()` en una cola que ya estaba vacía, no tendrá ningún efecto sobre las tareas que estén esperando recibir datos en ella (seguirán bloqueadas).

¿Cuál es el efecto de las prioridades de las Tareas al escribir y leer en una Cola ?

En la API nativa de FreeRTOS, las prioridades de las tareas dictan de forma matemática y estricta el orden en que el sistema operativo gestiona el flujo de datos. El *scheduler* es puramente determinista y expulsivo (*preemptive*): **la tarea lista para ejecutarse con la mayor prioridad siempre tomará el control de la CPU de manera inmediata**.

Cuando aplicamos esto a las colas, las diferencias de prioridad alteran drásticamente el comportamiento del sistema. A continuación se detallan los dos escenarios principales.

1. Tarea Lectora (Consumidora) con MAYOR prioridad que la Escritora (Productora)

Este es uno de los patrones de diseño más limpios y predecibles en sistemas de tiempo real.

¿Qué sucede internamente?

1. La **Tarea Lectora (Alta Prioridad)** intenta leer la cola usando `xQueueReceive()`. Como la cola está vacía, se bloquea inmediatamente y cede la CPU.
2. La **Tarea Escritora (Baja Prioridad)** toma el control de la CPU y genera un dato.
3. Al ejecutar `xQueueSend()`, el kernel copia el dato a la cola y detecta que hay una tarea de mayor prioridad esperando por él.
4. **Expulsión inmediata:** El *scheduler* suspende a la tarea escritora a mitad de su ejecución y le entrega la CPU a la lectora de forma instantánea.
5. La tarea lectora despierta, consume el dato, vacía la cola, intenta leer el siguiente, se vuelve a bloquear (porque ya no hay más datos) y la CPU regresa a la tarea de baja prioridad para que continúe.

Efecto en la Cola:

- **La cola siempre está vacía:** La cola nunca acumula más de un elemento a la vez. Funciona prácticamente como un canal de paso directo.
- **Latencia mínima:** El tiempo que pasa desde que se produce el dato hasta que se procesa es el mínimo físicamente posible del sistema (tiempo de cambio de contexto).

2. Tarea Escritora (Productora) con MAYOR prioridad que la Lectora (Consumidora)

Este escenario requiere mucha atención en el diseño, ya que puede provocar que la cola se llene rápidamente.

¿Qué sucede internamente?

1. La **Tarea Escritora (Alta Prioridad)** genera datos a su propio ritmo y los envía a la cola con `xQueueSend()`.
2. Como es la tarea más prioritaria, **no suelta la CPU** tras enviar el dato; continúa ejecutándose y enviando más elementos uno tras otro.
3. La **Tarea Lectora (Baja Prioridad)** permanece en estado *Ready* (Lista), pero no puede ejecutarse porque el procesador está ocupado con la tarea de alta prioridad.
4. **Bloqueo por cola llena:** Tarde o temprano, la tarea escritora llenará la cola hasta su límite (`uxQueueLength`). En el siguiente `xQueueSend()`, si se configuró un tiempo de espera (*Block Time*), la tarea escritora finalmente se bloqueará.

5. Al bloquearse la escritora, la **Tarea Lectora (Baja Prioridad)** toma el control de la CPU, extrae **un solo dato** de la cola (liberando una ranura).
6. En el instante en que se libera esa ranura, el kernel despierta a la escritora de alta prioridad, expulsando inmediatamente a la lectora de la CPU.

Efecto en la Cola:

- **La cola siempre está llena:** El almacenamiento operará al límite de su capacidad.
- **Procesamiento en ráfagas:** La tarea lectora solo logra avanzar un paso cada vez que la cola se satura por completo.

3. ¿Qué pasa si MÚLTIPLES tareas se bloquean en la misma Cola?

FreeRTOS resuelve los conflictos de concurrencia de múltiples tareas utilizando estrictamente las prioridades asignadas:

Al leer (Varias tareas esperando en una cola vacía):

Cuando una interrupción o una tarea secundaria deposita un dato en la cola, el kernel revisa la lista de tareas bloqueadas por lectura (`xTasksWaitingToReceive`):

- **Prioridades diferentes:** Despierta única y exclusivamente a **la tarea que tenga la prioridad más alta**, sin importar cuánto tiempo lleven esperando las demás.
- **Prioridades iguales:** Si dos tareas con la misma prioridad compiten por el dato, el kernel despierta a **la tarea que lleve más tiempo bloqueada** (orden FIFO de llegada al bloqueo).

Al escribir (Varias tareas intentando meter datos en una cola llena):

Cuando una tarea consumidora retira un dato y abre un espacio vacío, el kernel revisa la lista de tareas bloqueadas por escritura (`xTasksWaitingToSend`):

- Despierta inmediatamente a **la tarea escritora de mayor prioridad** para que copie su información.
- Si tienen la misma prioridad, le concede el acceso a **la que esperó primero**.